

App Development Project Report

App Dev - 2 Project Report

1. Student Details

Name: Sreeman.M

RollNumber: 24f1000587

Email: 24f1000587@ds.study.iitm.ac.in

About Me: I am naturally curious and enjoy understanding how things truly work, especially in technology and problem-solving. I like learning step by step, asking questions, and turning complex ideas into something clear and practical.

2. Project Details

Project Title: Placement Portal Application (PlaceTrack)

Problem Statement:

Institutes currently manage campus recruitment through spreadsheets, emails, and manual coordination, making it hard to track company approvals, placement drives, student registrations, and applications. This project required building a Placement Portal Application (PPA) where Admin, Companies, and Students interact through role-based access to streamline the entire placement process.

Approach:

I built PlaceTrack — a Flask + VueJS application with a unified role-based user model, where JWT authentication and decorators enforce that admins approve all companies/drives before they go live, keeping the workflow trustworthy and auditable. Redis/Celery power the three required background jobs (deadline reminders, monthly HTML/PDF reports emailed via SMTP, async CSV export), while Bootstrap-based Vue components deliver a distinctive, fully responsive UI built around a "track rail" visual metaphor for an application's journey from Applied to Selected.

3. AI/LLM Declaration

I used **ChatGPT (GPT-5)**, **Claude (Sonnet-5)** to assist in writing SQLAlchemy model definitions, creating API documentation samples, Docker setup and improving variable naming consistency. The extent of AI/LLM usage is around **10–15%**, limited to **code suggestions and documentation formatting**. All final implementation logic, debugging, and integration were done manually

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework
SQLAlchemy	Object Relational Mapper for SQLite database
Flask-JWT-Extended	Generates and verifies JWT tokens for authentication and role-based access control
Flask-Caching	Caches expensive read queries (dashboard stats, drive listings) using Redis, with timeout-based expiry
Flask-CORS	Allows the frontend to make cross-origin API requests safely
VueJS 3	Frontend framework — builds the reactive, component-based single-page application (SPA)
Redis	In-memory store used both as the cache backend and as the Celery message broker/result backend
Celery	Distributed task queue — runs background jobs (reminders, reports, exports) outside the main request-response cycle
Jinja2	Template engine for rendering dynamic HTML pages

Bootstrap 5	Frontend styling and responsive design
Chart.js	To visualize trends such as activity vs. time
Flask-Login	User authentication and session management
WTForms	Frontend form validation
SQLite	Lightweight local database for storing user data

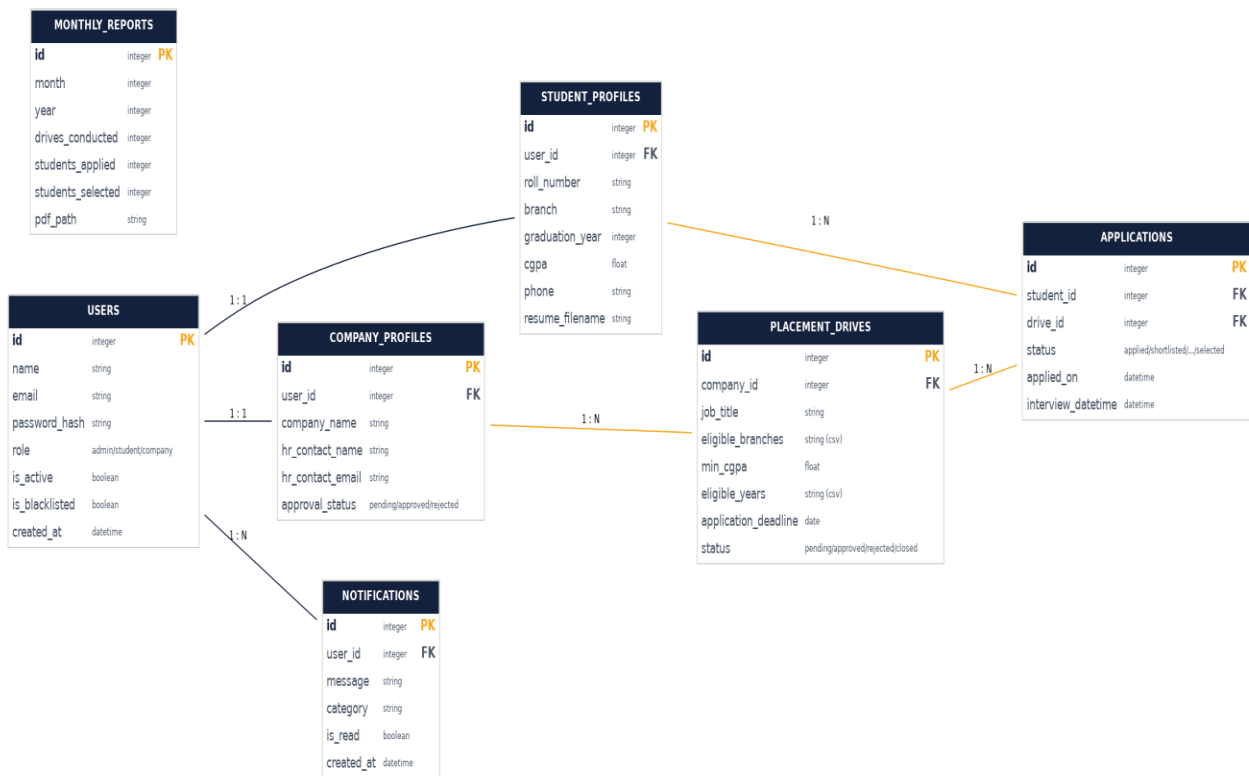
5. Database Schema / ER Diagram

Tables:

1. **Users** — The unified identity for every account — name, email, hashed password, and a `role` field that determines access. Also tracks **is_active** and **is_blacklisted** for admin moderation.
2. **Student_profiles** — Student-specific details linked 1:1 to a `USERS` row — roll number, branch, graduation year, CGPA, phone, and the uploaded resume filename. **Activity Type** — stores predefined activity categories (id, name, description)
3. **Company_profiles** — Company-specific details linked 1:1 to a `USERS` row — company name, HR contact info, and **approval_status** (pending/approved/rejected), since companies need admin sign-off before posting drives.
4. **Placement_drives** — Each recruitment drive posted by a company — job title/description, eligibility criteria (branches, min CGPA, graduation years), application deadline, package, and its own approval **status**.
5. **Applications** — The link between a student and a drive they applied to — tracks **status** (applied → shortlisted → interview scheduled → selected/rejected), applied date, interview time, and whether an offer letter has been generated.

6. **Notifications** — In-app notifications for the bell icon — one row per message sent to a user (approvals, status changes, reminders), with a read/unread flag.
7. **Monthly Reports** — A log of every auto-generated monthly activity report — drives conducted, students applied/selected that month, and the path to the generated PDF. Standalone table, no foreign keys.

ER- DIAGRAM :



6. API Resource Endpoints

Endpoint	Method	Description
/api/auth/login	POST	Unified login for all roles
/api/student/drives/{id}/apply	POST	Student applies to drive

YAML API Definition File:

Included separately in the submission ZIP as `api.yaml`.

7. Video Presentation

Drive Link:

https://drive.google.com/file/d/11CX_eMkNam7ofagRp_W7oGmktMS1k2mp/view?usp=sharing

(Accessible to all with "View" permission.)
